

Indian Institute of Technology (IIT-Bombay)

AUTUMN Semester, 2026

COMPUTER SCIENCE AND ENGINEERING

CS230: Digital Logic Design + Computer Architecture

Mid Semester Examination

Full Marks: 35

Time allowed: 2.0 hours

INSTRUCTIONS: Do not make any extra assumptions other than those stated in the questions. The marks will only be given if a precise, sound, and clearly written explanation is provided that is understandable by the examiners. Any extra unnecessary writing, verbal arguments based on that, and unnecessary crib will result in a reduction of 5 marks.

Roll Number: _____

Name: _____

QN	1	2	3	4	5	6	7	Total
Marks								

1. The main difference between SR flip-flop and JK flip-flop is:
 - (a) SR flip-flop does not have a clock while JK does.
 - (b) SR flip-flop is edge-triggered while JK is level-triggered
 - (c) SR flip-flop has invalid states while JK has no invalid states
 - (d) SR flip-flop is level-triggered while JK flip-flop is edge triggered.

Just mark the correct answer. No explanation needed.

(1 mark)

Solution: (c) SR flip-flop has invalid states while JK has no invalid states.

1 mark only for the correct answer. 0 for wrong answer.

2. 2's complement representation is preferred over 1's complement representation, because:

- (a) it has only one representation of zero.
- (b) 2's complement requires fewer bits to represent every signed integer.
- (c) 2's complement can represent twice as many positive integers as 1's complement.
- (d) None of the above.

Just mark the correct answer. No explanation needed.

(1 mark)

Solution: (a) it has only one representation of zero.

1 mark only for the correct answer. 0 for wrong answer.

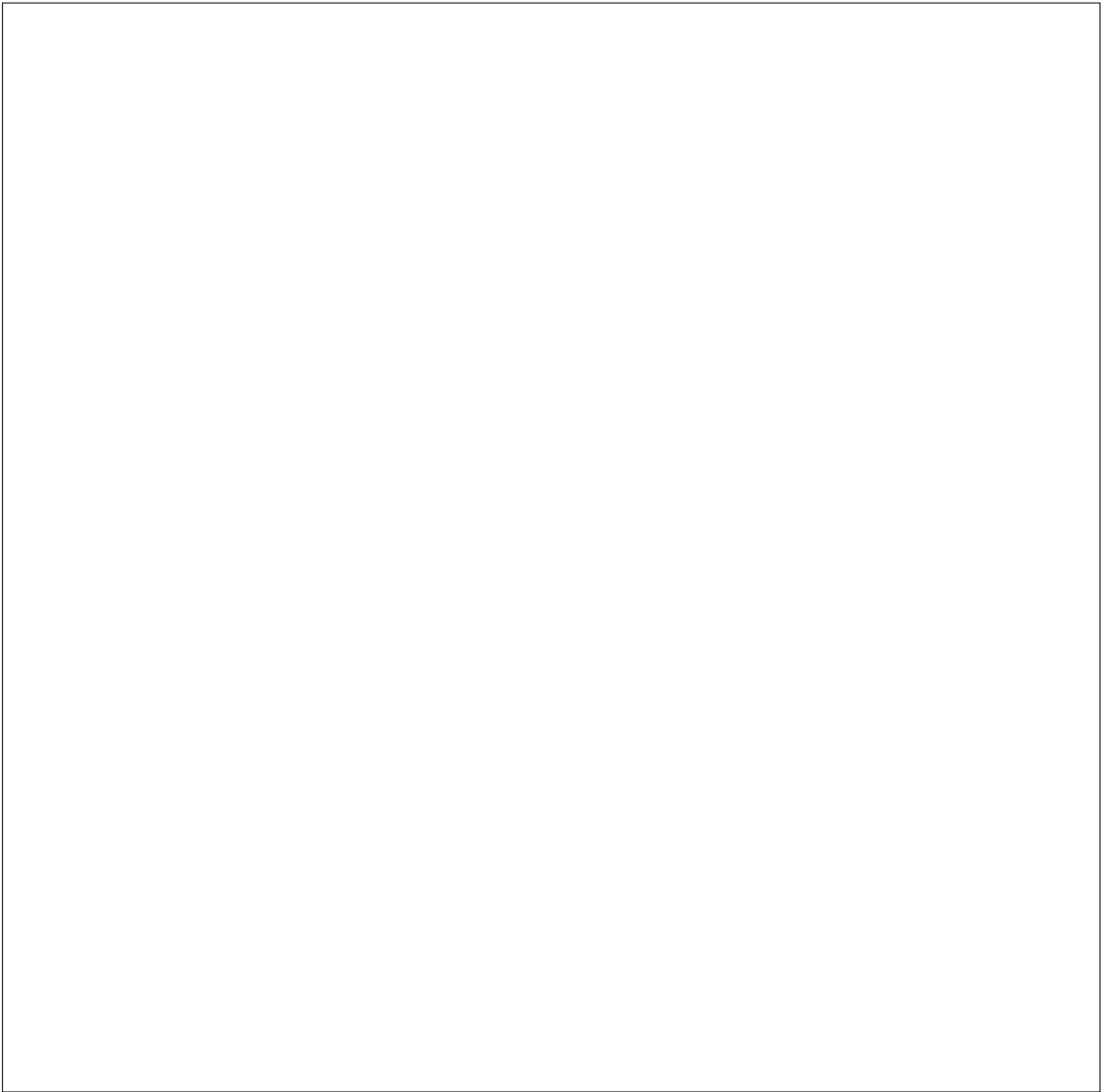
3. Suppose you have 45-bit instructions in a MIPS like ISA, which has 64 registers. The size of the **shamt** and **funct** fields for the R-type instructions are 5 and 6 bits, respectively. What are sizes of the 1) opcode field; 2) immediate field in I type instructions; 3) branch address field in the J type instructions. (3 marks)

Solution: $|Regt| + |Regs1| + |Regs2| + |shamt| + |funct| = 6 + 6 + 6 + 5 + 6 = 29$ bits.

1) $op = 45 - 29 = 16$. 2) $|Immediate| = 45 - (16(op) + 6(Regt) + 6(Regs1)) = 17$. 3)

$|branch\ address\ field| = 45 - 16|op| = 29$

1 mark for each correct answer.



4. Assuming MIPS ISA, suppose the program counter (PC) is set to 0x2000 0000.
- (a) Can you jump to 0x4000 0000 using **a single j** instruction? Answer in YES or NO. (2 marks)
 - (b) What is the address range you can reach from 0x2000 0000 using **a single j** instruction? Explain your answer with proper calculations. (3 marks)
 - (c) Can you jump to 0x4000 0000 using **a single beq** instruction? Answer in YES or NO. (2 marks)
 - (d) What is the address range you can reach from 0x2000 0000 using **a single beq** instruction? Explain your answer with proper calculations. (3 marks)

- (e) How can you transfer control from address 0x2000 0000 to address 0x1000 0000? You cannot use **beq**, **bne**, and you may use **at most one** control-transfer instruction for the transfer. Give a valid MIPS assembly sequence and justify how the target address is reached. (5 marks)

Solution:

- (a) No. To see why, check the next answer.
(b) The target address of a MIPS j instruction is calculated as

$$\text{Target Address} = \{(PC + 4)[31 : 28], \text{instr}[25 : 0], 00\}.$$

Given

$$PC = 0x20000000,$$

we have

$$PC + 4 = 0x20000004.$$

Therefore, the upper 4 bits of the target address are fixed to

$$(PC + 4)[31 : 28] = 0x2.$$

The remaining 26 bits are obtained from the target field of the j instruction, while the least significant 2 bits are always 00.

For the minimum possible target address, all 26 target bits are zero:

$$\begin{aligned} \text{Target}_{\min} &= \{0010, \underbrace{00 \dots 00}_{26 \text{ bits}}, 00\} \\ &= 0x20000000. \end{aligned}$$

For the maximum possible target address, all 26 target bits are one:

$$\begin{aligned} \text{Target}_{\max} &= \{0010, \underbrace{11 \dots 11}_{26 \text{ bits}}, 00\} \\ &= 0x2FFFFFFC. \end{aligned}$$

Hence, the address range reachable using a j instruction is

$$\boxed{0x20000000 \leq \text{Target Address} \leq 0x2FFFFFFC}$$

where the reachable addresses are spaced by 4 bytes because MIPS instructions are word-aligned.

The size of this address region is

$$2^{26} \times 4 = 2^{28} \text{ bytes} = 256 \text{ MB.}$$

Therefore, the address `0x40000000` cannot be reached using a single `j` instruction from `0x20000000`.

(c) NO. To see why, check the next answer.

(d) For a MIPS `beq` instruction, the target address is calculated as

$$\text{Target Address} = (PC + 4) + (\text{SignExt}(\text{offset}_{16}) \ll 2).$$

Given

$$PC = 0x20000000,$$

we have

$$PC + 4 = 0x20000004.$$

The `beq` instruction contains a 16-bit signed offset. Therefore,

$$-2^{15} \leq \text{offset}_{16} \leq 2^{15} - 1,$$

or equivalently,

$$-32768 \leq \text{offset}_{16} \leq 32767.$$

Since the offset is shifted left by 2 bits, the actual byte displacement is

$$\text{Displacement} = \text{offset}_{16} \times 4.$$

For the minimum possible target address, we use the most negative offset, $\text{offset}_{16} = -32768$:

$$\begin{aligned} \text{Target}_{\min} &= 0x20000004 + (-32768 \times 4) \\ &= 0x20000004 - 131072 \\ &= 0x20000004 - 0x00020000 \\ &= \boxed{0x1FFE0004}. \end{aligned}$$

For the maximum possible target address, we use the largest positive offset, $\text{offset}_{16} = 32767$:

$$\begin{aligned}
\text{Target}_{\max} &= 0x20000004 + (32767 \times 4) \\
&= 0x20000004 + 131068 \\
&= 0x20000004 + 0x0001FFFC \\
&= \boxed{0x20020000}.
\end{aligned}$$

Hence, the address range reachable using a `beq` instruction from `0x20000000` is

$$\boxed{0x1FFE0004 \leq \text{Target Address} \leq 0x20020000}$$

where the reachable addresses are spaced by 4 bytes because MIPS instructions are word-aligned.

Therefore, the address `0x40000000` cannot be reached using a single `beq` instruction from `0x20000000`.

(e) To reach the address

`0x10000000`

from

`0x20000000`,

without using `beq`, `bne`, we can first place the destination address in a register and then use the `jr` (jump register) instruction.

Method 1: Using `lui` and `jr`

Since the lower 16 bits of `0x10000000` are zero, the target address can be constructed directly using `lui`:

```

lui  $t0, 0x1000    # $t0 = 0x10000000
jr   $t0            # PC = 0x10000000

```

The `lui` instruction loads the immediate value into the upper 16 bits of the register. Therefore,

$$\$t0 = 0x1000 \ll 16 = 0x10000000.$$

The instruction

`jr $t0`

then sets the program counter to the value stored in register `$t0`:

$$PC \leftarrow \$t0 = \boxed{0x10000000}.$$

Method 2: Using `addi`, `sll`, and `jr`

The target address can also be constructed using an arithmetic instruction followed by a shift:

```
addi $t0, $zero, 0x1000    # $t0 = 0x00001000
sll  $t0, $t0, 16          # $t0 = 0x10000000
jr   $t0                   # PC = 0x10000000
```

After the `addi` instruction,

$$\$t0 = 0x00001000.$$

The `sll` instruction shifts this value left by 16 bits:

$$\$t0 = 0x00001000 \ll 16 = 0x10000000.$$

Therefore,

```
jr $t0
```

transfers control to

0x10000000.

Method 3: Loading the Target Address from Memory

If the value `0x10000000` is already stored in memory, it can first be loaded into a register and then used with `jr`:

```
lw  $t0, 0($t1)           # Assume memory at $t1 contains 0x10000000
jr  $t0                   # PC = 0x10000000
```

After the `lw` instruction,

$$\$t0 = 0x10000000.$$

Hence,

$$PC \leftarrow \$t0 = \text{\texttt{0x10000000}}.$$

A direct `j` instruction cannot be used to reach `0x10000000`. For a MIPS `j` instruction, the target address is constructed as

$$\text{Target} = \{(PC + 4)[31 : 28], \text{instr}[25 : 0], 00\}.$$

For

$$PC = 0x20000000,$$

we have

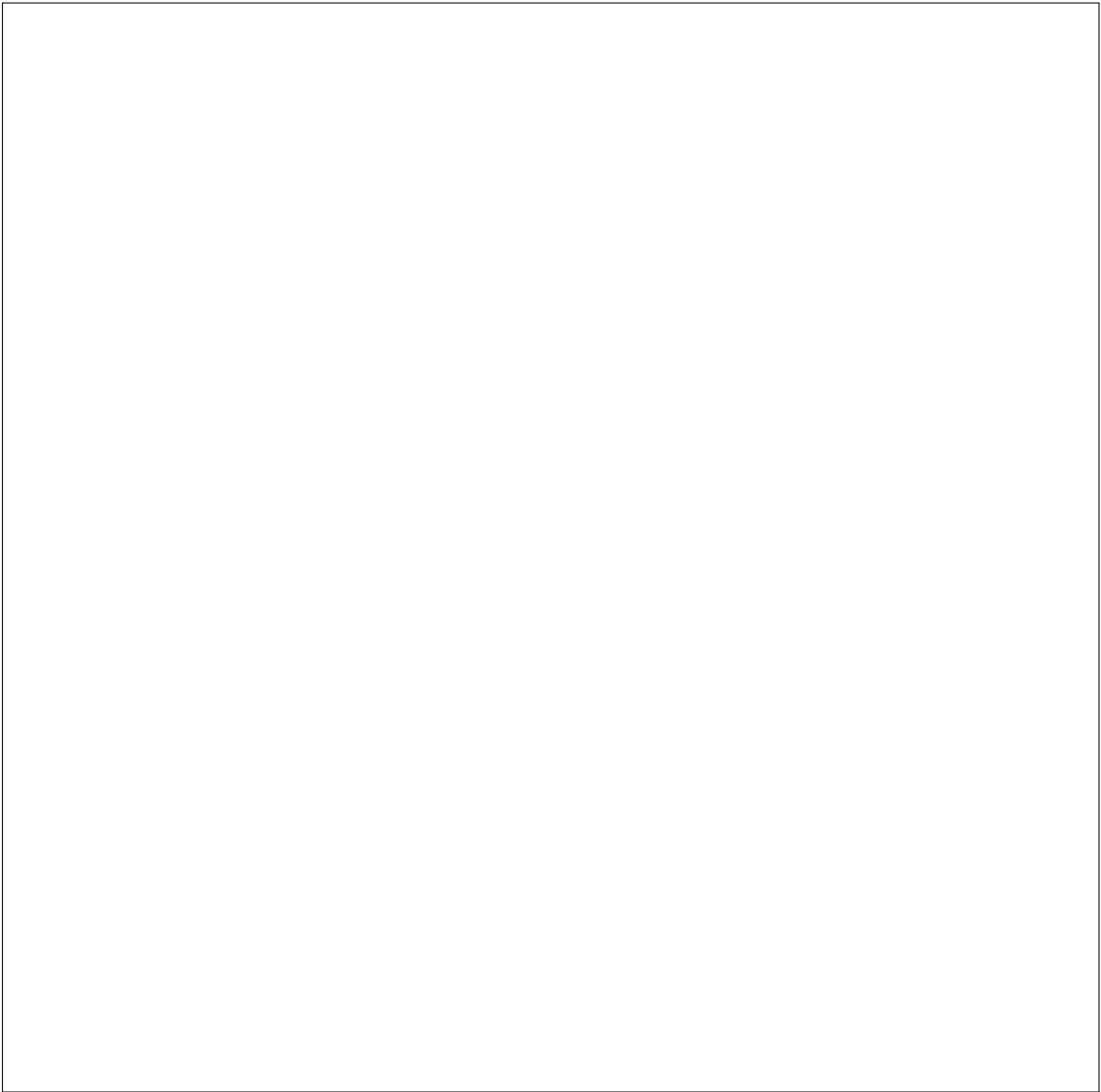
$$PC + 4 = 0x20000004.$$

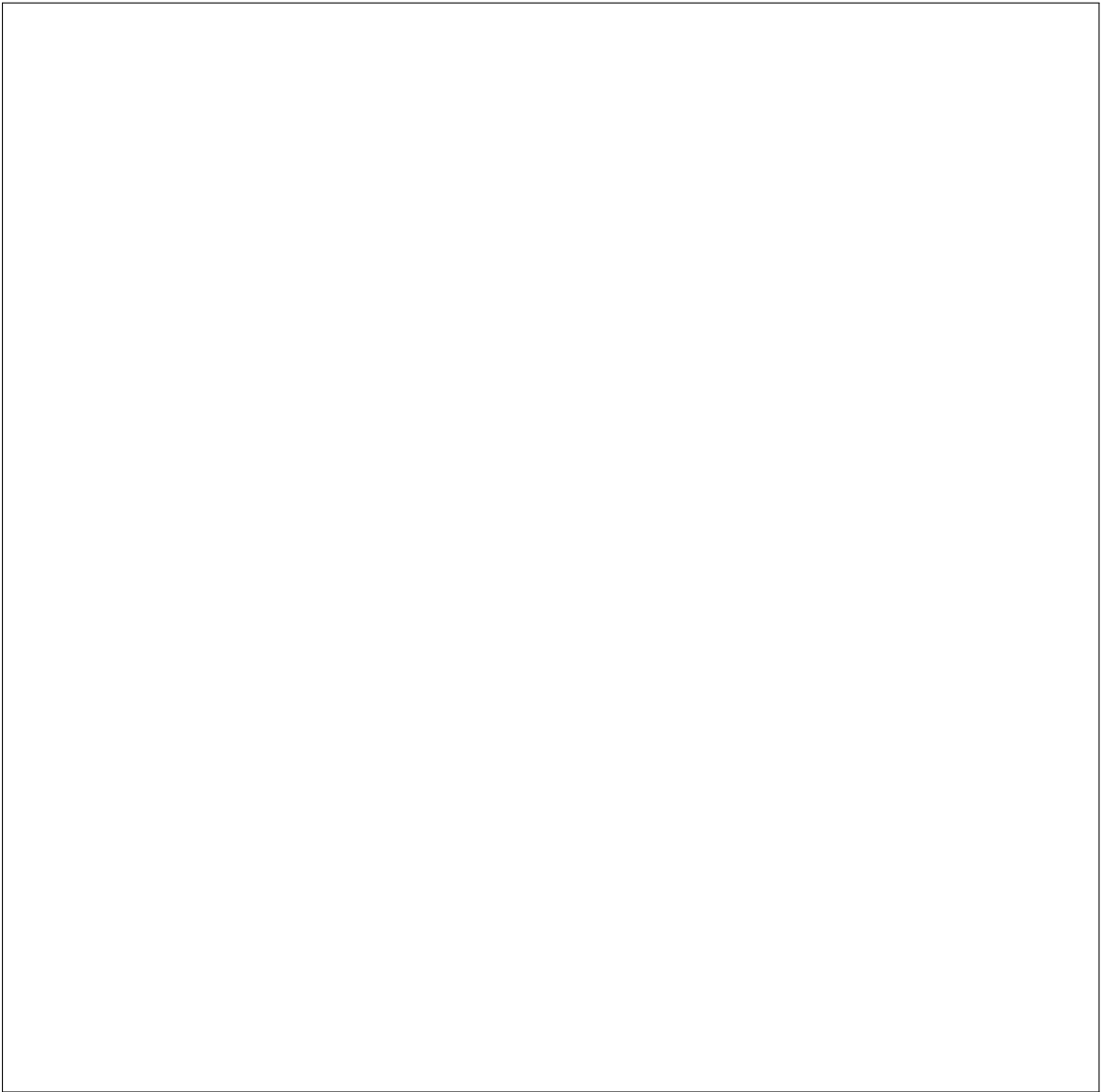
Thus, the upper four bits of the target address are fixed to 0x2. Consequently, a direct j instruction can only reach addresses in the range

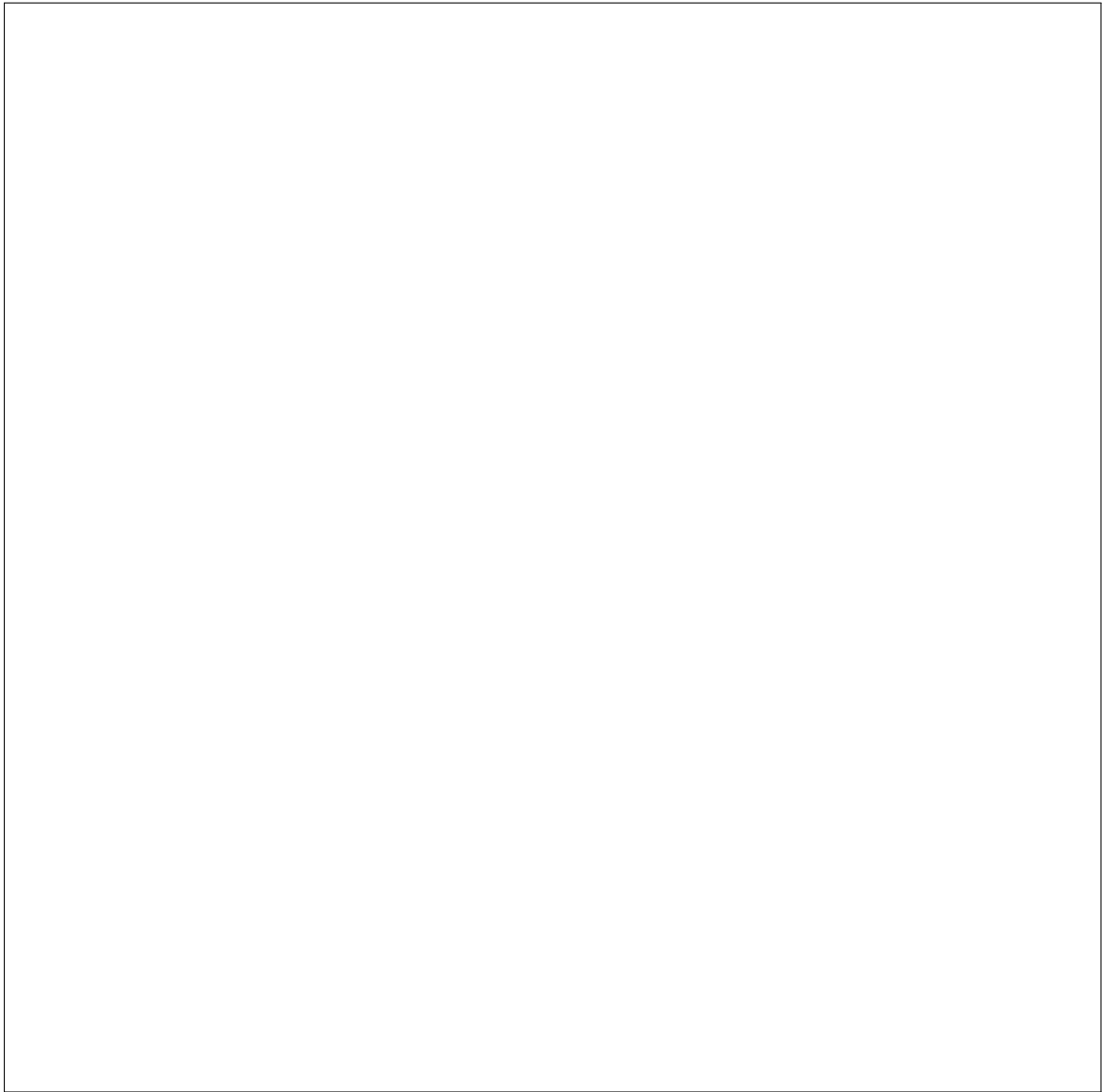
$$0x20000000 \leq \text{Target} \leq 0x2FFFFFFC.$$

Therefore, 0x10000000 cannot be reached directly using j. The general solution is to load or construct 0x10000000 in a register and use jr to transfer control to that address.

Grading Scheme: for (a), (c), give marks only if correct. For (b) and (d) give 1 if the target address formula is correct. Give 3 if the address calculations are correct. For (e), there can be multiple options. Give marks if someone performs the address calculation and uses \$jr.







5. Consider the following C function:

```
int f(int a, int b, int c, int d)
{
    return func(func(a,b), c+d);
}
```

Assume that the function declaration for `func` is

```
int func(int a, int b);
```

- (a) Translate the function **f** into MIPS assembly language using the standard MIPS calling convention. You may use caller-saved temporary registers **\$t0** through **\$t7**, and caller-saved argument registers **\$a0**–**\$a4**, but you must not use any callee-saved registers (**\$s0**–**\$s7**). Assume that the input parameters are initially available as **\$a0 = a**, **\$a1 = b**, **\$a2 = c**, and **\$a3 = d**. If temporary registers are required, use the lower-numbered registers first. Also, do not unnecessarily save any registers. (3 marks)
- (b) Tail-call optimization (TCO) is an optimization that can be applied when a function call is the last operation performed by the current function. In such a case, instead of making a regular function call and subsequently returning to the caller, control can be transferred directly to the called function without creating an additional return step. The called function (called as “**j func**” rather than “**jal func**”) then returns directly to the original caller. Apply tail-call optimization to the MIPS assembly implementation of **f**. Give the complete optimized MIPS assembly code and briefly explain why the optimization is valid. (2 marks)

Solution:

- (a) According to the standard MIPS calling convention, the four arguments *a*, *b*, *c*, and *d* are initially available in registers **\$a0**, **\$a1**, **\$a2**, and **\$a3**, respectively.

The value $c + d$ is required after the first call to **func**. Since the temporary registers **\$t0**–**\$t7** are caller-saved, a value stored in one of these registers cannot be assumed to survive a call to **func**. Therefore, $c + d$ is saved on the stack before the first function call.

Moreover, **f** is a non-leaf function. Since **jal** overwrites **\$ra**, the original value of **\$ra** must also be saved on the stack.

A valid MIPS implementation is:

```
f:
    addiu $sp, $sp, -8      # Allocate stack space
    sw     $ra, 4($sp)      # Save return address

    add    $t0, $a2, $a3     # $t0 = c + d
    sw     $t0, 0($sp)      # Preserve c+d across the first call

    # First call: func(a,b)
    # $a0 = a and $a1 = b
    jal    func
    nop

    # $v0 = func(a,b)
    move   $a0, $v0         # First argument = func(a,b)
    lw     $a1, 0($sp)      # Second argument = c+d

    # Second call: func(func(a,b),c+d)
```

```

jal    func
nop

# $v0 already contains the return value of f
lw     $ra, 4($sp)      # Restore return address
addiu  $sp, $sp, 8      # Restore stack pointer

jr     $ra
nop

```

After the first call,

$$\$v0 = \text{func}(a, b).$$

Therefore, the arguments for the second call are set as

$$\$a0 = \text{func}(a, b), \quad \$a1 = c + d.$$

The second call consequently computes

$$\text{func}(\text{func}(a, b), c + d).$$

Its return value is already stored in $\$v0$, which is also the required return value of f .

- (b) The first call to `func` cannot be tail-call optimized because `f` must continue executing after the call in order to prepare the arguments for the second call.

However, the second call

$$\text{func}(\text{func}(a, b), c + d)$$

is a tail call because its return value is immediately returned by `f`. Therefore, instead of calling `func` using `jal` and subsequently returning using `jr $ra`, `f` can restore its stack frame and directly jump to `func`.

The tail-call optimized implementation is:

```

f:
    addiu $sp, $sp, -8      # Allocate stack space
    sw    $ra, 4($sp)      # Save original return address

    add   $t0, $a2, $a3     # $t0 = c+d
    sw    $t0, 0($sp)      # Preserve c+d across the first call

    # First call: func(a,b)
    jal   func

```

```

nop

# Prepare arguments for the second call
move  $a0, $v0      # $a0 = func(a,b)
lw     $a1, 0($sp)   # $a1 = c+d

# Restore f's stack frame
lw     $ra, 4($sp)   # Restore original return address
addiu  $sp, $sp, 8   # Restore stack pointer

# Tail call
j      func
nop

```

The original value of `$ra` must be restored before the `j func` instruction. The first `jal func` overwrites `$ra` with a return address inside `f`. Restoring `$ra` ensures that it again contains the return address of the original caller of `f`.

Since `j` does not modify `$ra`, when the tail-called `func` eventually executes

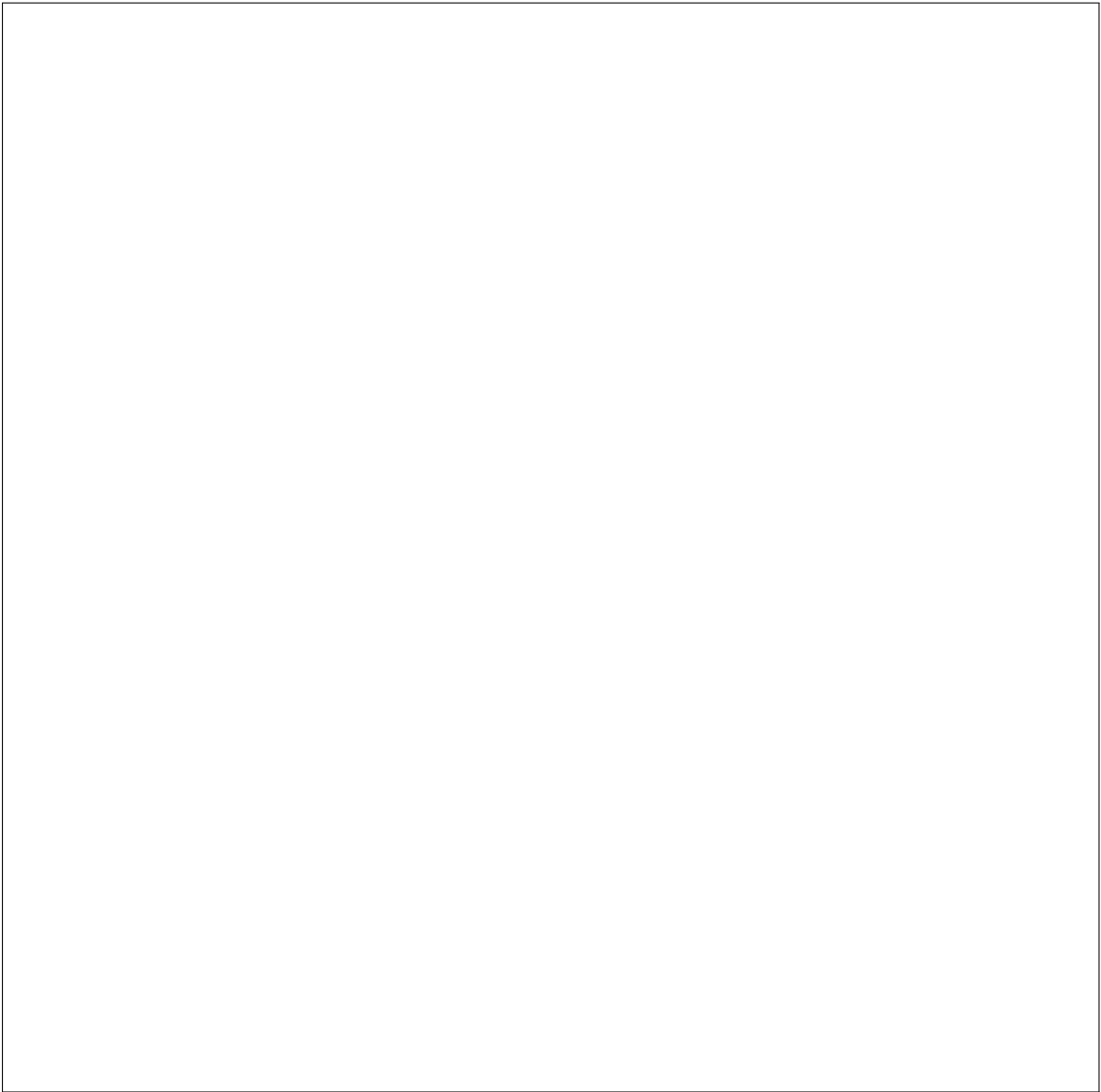
```
jr $ra
```

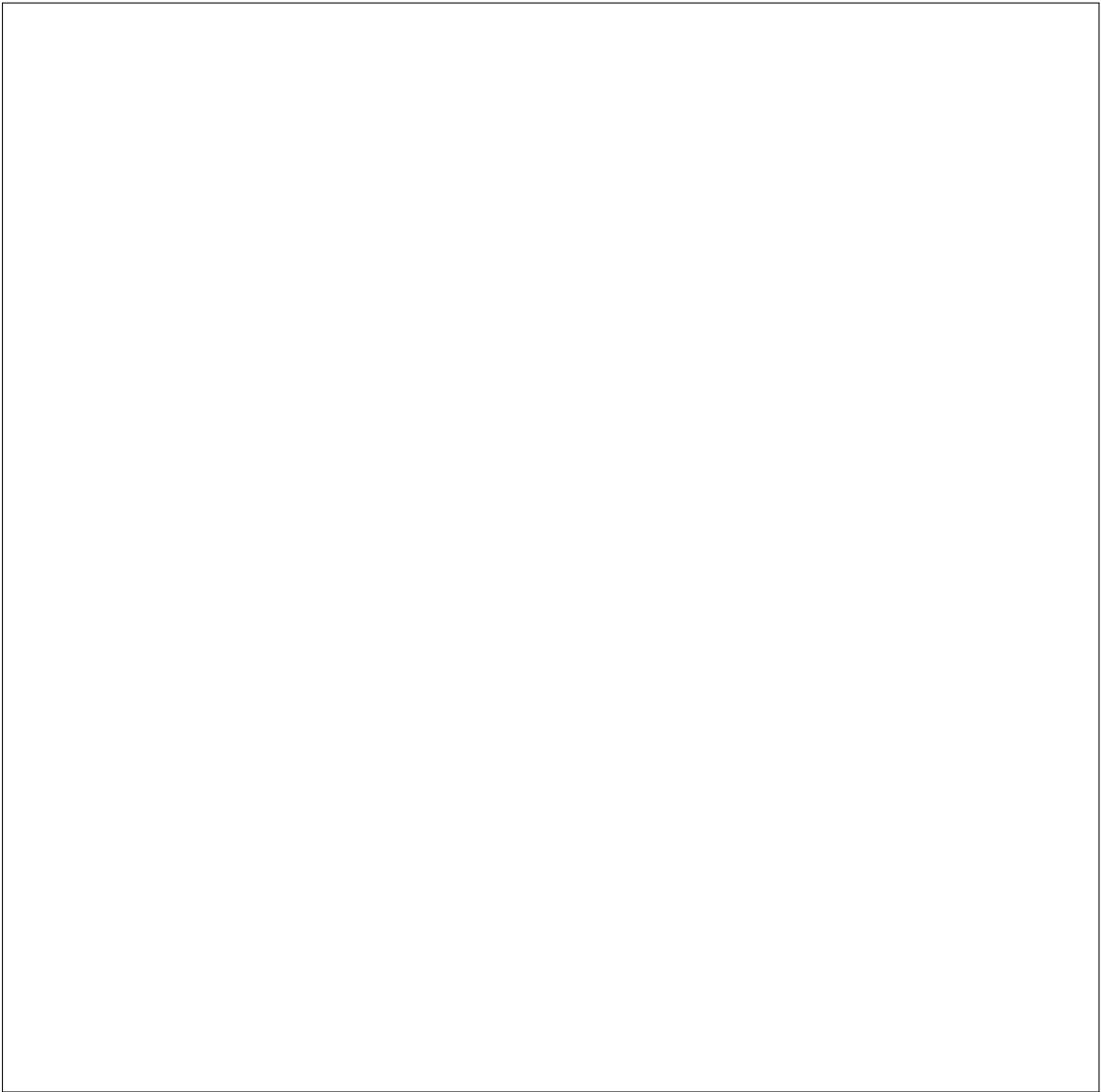
it returns directly to the original caller of `f`. Thus, the second function call and the subsequent return from `f` are replaced by a single tail jump.

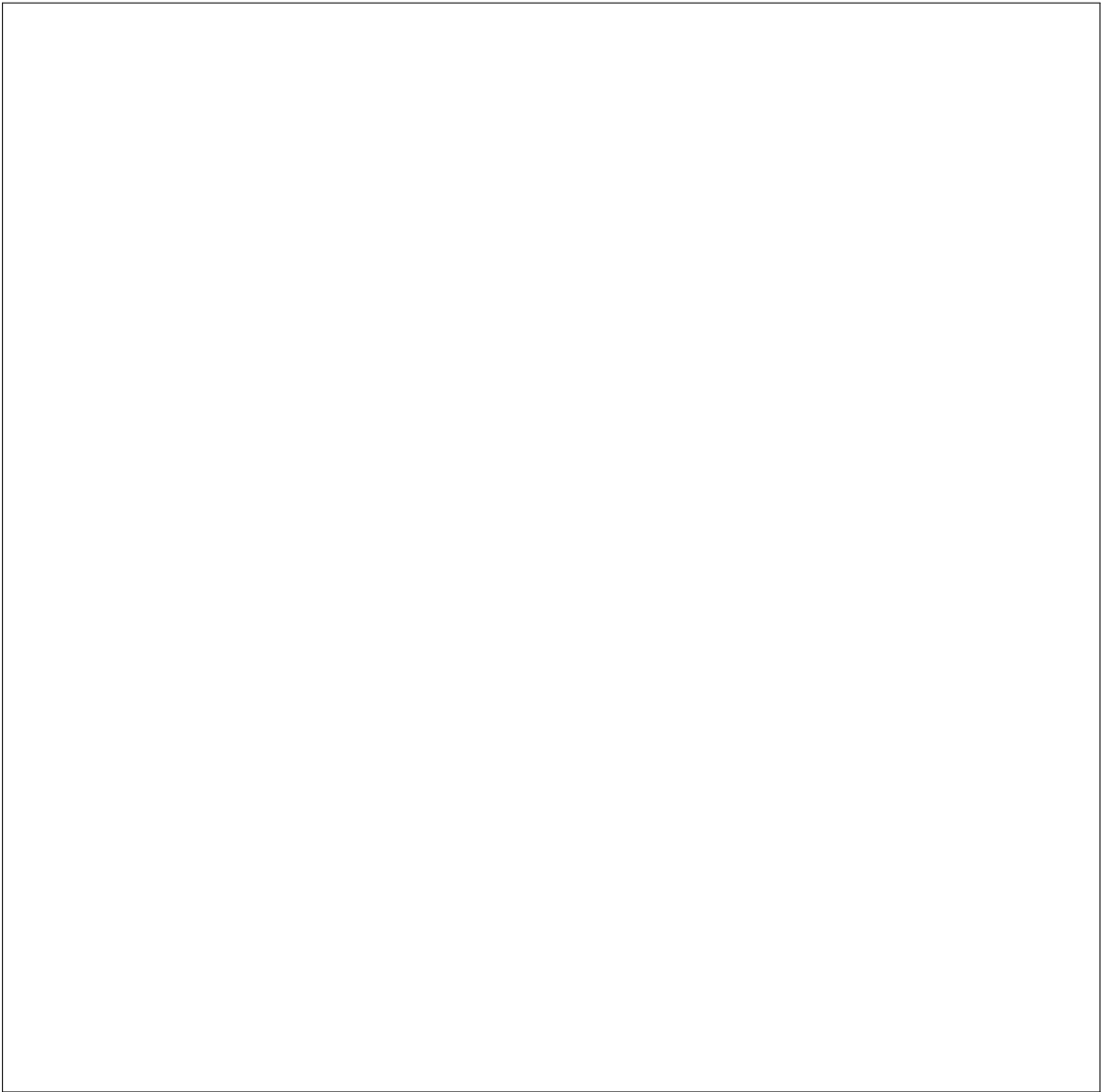
Grading Scheme: for part 1, only check if the code is at least partially correct. Main point to check is if they save necessary caller saved registers before function call. The answer and the registers to be saved might differ according to how they code it. But one thing must be clear – any caller saved register that must retain its value even after a function call must be saved. Give full marks if they have got it correct and the function looks semantically ok.

For part 2, mainly check if they have restored the `$ra` before `j func` call. Give full marks if they have got this correct.









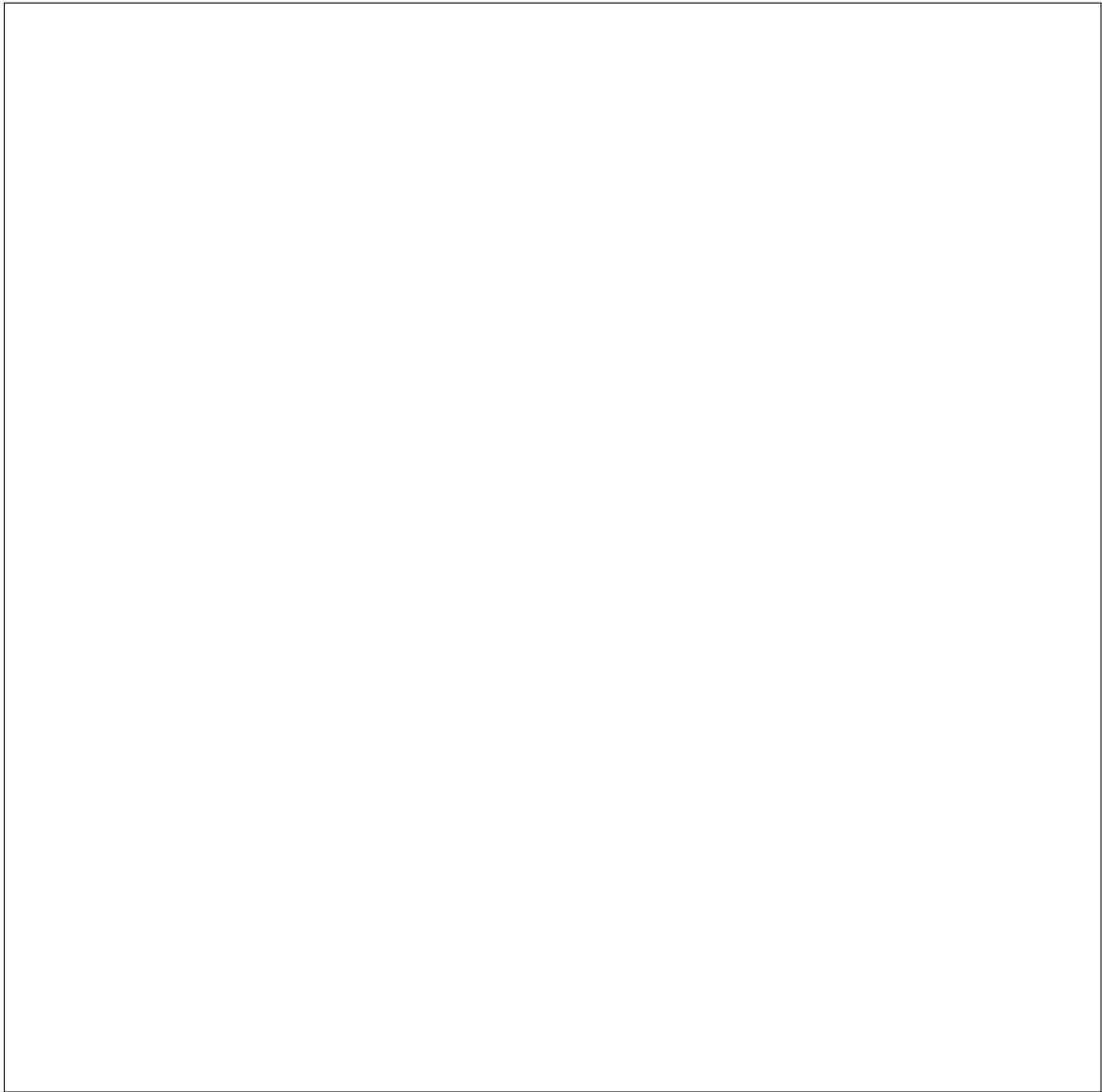
6. Construct the following function using a 4×1 MUX.

$$f(A, B, C) = \sum(1, 3, 5, 6)$$

- . Show all necessary tables (especially ones used to construct MUX based circuits) and the block diagram. (5 marks)

Solution: Refer to slide number 14 in "Combinational Circuits" lecture.

Grading Scheme: Give 3 marks if the MUX table has been constructed correctly. Give 4 marks if the expressions have been found correctly. Give 5 if everything is correct.



7. Consider that you are given a synchronous mod 2 counter and a synchronous mod 5 counter. Both counters have an active-high enable input. When the enable input of a counter is LOW, the counter retains its current state. Both counters are positive-edge triggered.

Using only these two counters and combinational logic, construct a synchronous mod 10 counter. Both counters must be driven by the same clock.

1. Enumerate the states of the resulting mod 10 counter in terms of the states of the *mod* 5 and the *mod* 2 counter in the required counting sequence. (2 marks)
2. Draw a block diagram showing the interconnection of the two counters. (3 marks)

Solution: Let the state of the synchronous mod 2 counter be denoted by Q_0 , and let the state of the synchronous mod 5 counter be denoted by Q_5 , where

$$Q_0 \in \{0, 1\}$$

$$Q_5 \in \{0, 1, 2, 3, 4\}.$$

Both counters are connected to the same clock.

For a mod 10 count, the mod 2 counter must advance on every clock edge. Hence,

$$\boxed{EN_2 = 1}.$$

The mod 5 counter should advance only when the mod 2 counter is currently in state 1. Therefore,

$$\boxed{EN_5 = Q_0}.$$

Thus, when $Q_0 = 0$, only the mod 2 counter advances, while when $Q_0 = 1$, both counters advance simultaneously on the next rising clock edge.

1. The required state sequence is

Decimal Count	Q_5	Q_0
0	0	0
1	0	1
2	1	0
3	1	1
4	2	0
5	2	1
6	3	0
7	3	1
8	4	0
9	4	1

Hence, the state transitions are

$$(0, 0) \rightarrow (0, 1) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (2, 0) \rightarrow (2, 1) \rightarrow (3, 0) \rightarrow (3, 1) \rightarrow (4, 0) \rightarrow (4, 1) \rightarrow (0, 0).$$

2. The block-level interconnection is

[TAs please add a block diagram](#)

Both counters receive the same clock. The output Q_0 of the mod 2 counter is connected only to the enable input of the mod 5 counter and not to its clock input. Therefore, both counters change state only

on the common rising clock edge, and the resulting mod 10 counter is synchronous.

Grading Scheme: For 1, give marks if one gets the idea of the (x, y) form of the state. For 2, give marks if the “enable” expressions are correct and the “clk” has been connected correctly. In the case of partial correctness give 1 or 2 depending upon how much is correct.

